Name: Rshijuta Pokharel

UID: U01122370

Email: pokharel.18@wright.edu

CEG7560

Visualization and Image Processing for Cyber Security

First Assignment Output Report

Task:

Install and/or familiarize yourself with the VTK framework using the provided lecture material as a guideline. The VTK website is a good resource for examples and test data. Create a visualization of geo locations of IP addresses on a globe. Use the VTK class vtkGeoAssignCoordinates to convert from longitude/lattitude to 3D locations similar to this example. You can also convert the coordinates manually yourself.

Implementation:

1. Loading and preparing the globe:

   `vtkOBJReader( )` was used to display the globe model. The bounds were extracted to find 'center' and 'radius', which is used to scaling the plotted points.

2. Reading and Filtering Dataset

```
37.386002 N/A 807
37.386002 N/A 807
116.388298 39.928902 0
116.388298 39.928902 0
-122.120598 47.680099 425
```

```python
try:
    lon, lat = float(parts[0]), float(parts[1])
    if -180 <= lon <= 180 and -90 <= lat <= 90:
        latitudes.append(lat)
        longitudes.append(lon)
except ValueError:
    print(f"Skipping invalid line: {line.strip()}")
    continue
```

   There were some invalid characters, ZIP codes and other anomalies in the *location.dat* file.
   Those were skipped.
   Only points within the given ranges `(-180 <= lon <= 180` *and* `-90 <= lat <= 90)` were stored in longitudes and latitudes list. This ensures only valid and correctly formatted geometric coordinates are plotted.

3. Converting 2D Coordinates to 3D space
   Following the trigonometric method described in [Link](), I manually converted each latitude and longitude into a 3D point on the sphere's surface. Each point was scaled by the globe's radius and inserted into a `vtkPoints( )` object.

```python
lat_rad = math.radians(lat)
lon_rad = math.radians(lon)

x = math.cos(lat_rad) * math.cos(lon_rad)
y = math.cos(lat_rad) * math.sin(lon_rad)
z = math.sin(lat_rad)
```

To check order of latitude and longitude, I tried this:

```
for i in range(5):
    print(f"Lon: {longitudes[i]}, Lat: {latitudes[i]}")
```

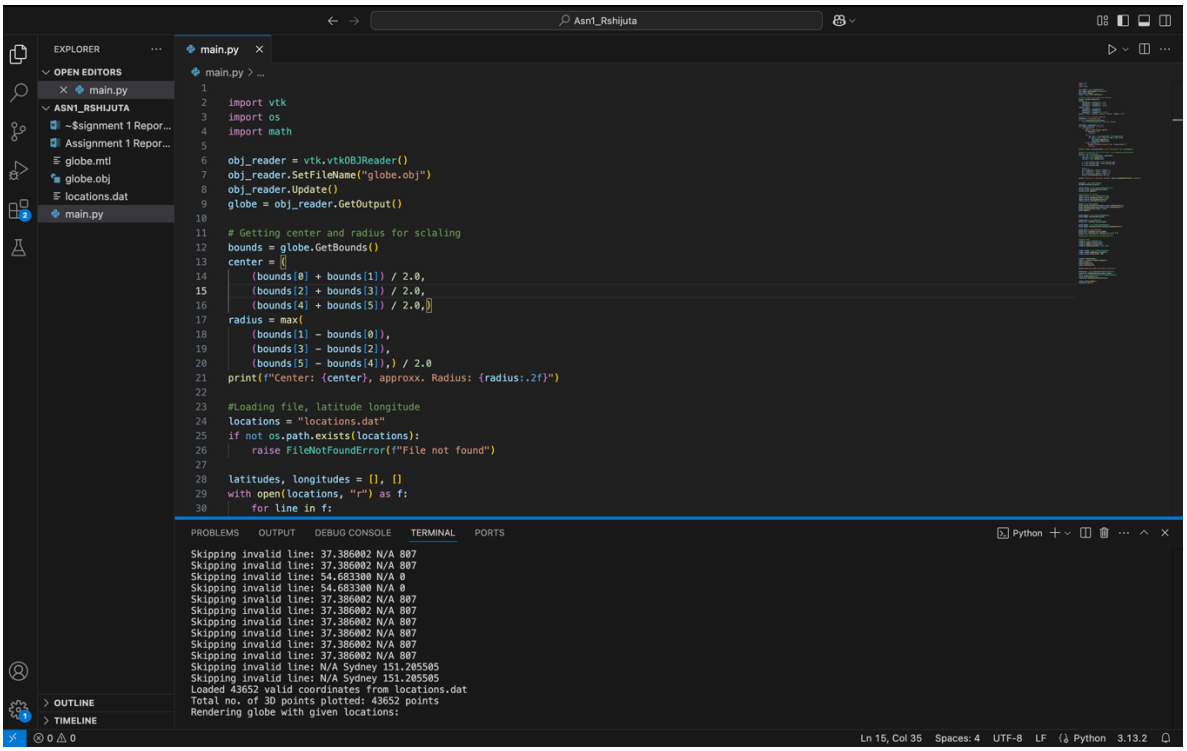
So, in code, the array is listed as:

```
lon, lat = float(parts[0]), float(parts[1])
```

```
rshiju11@Rshijutas-MacBook-Air-9 Asn1_Rshijuta % /us
Center: (13.373450000000002, 8.033050000000003, -17.
Loaded 43652 valid coordinates from locations.dat
Lon: 103.8000, Lat: 1.3667
Lon: 103.8000, Lat: 1.3667
Lon: 103.8000, Lat: 1.3667
Lon: 103.8000, Lat: 1.3667
Lon: 103.8000, Lat: 1.3667
```

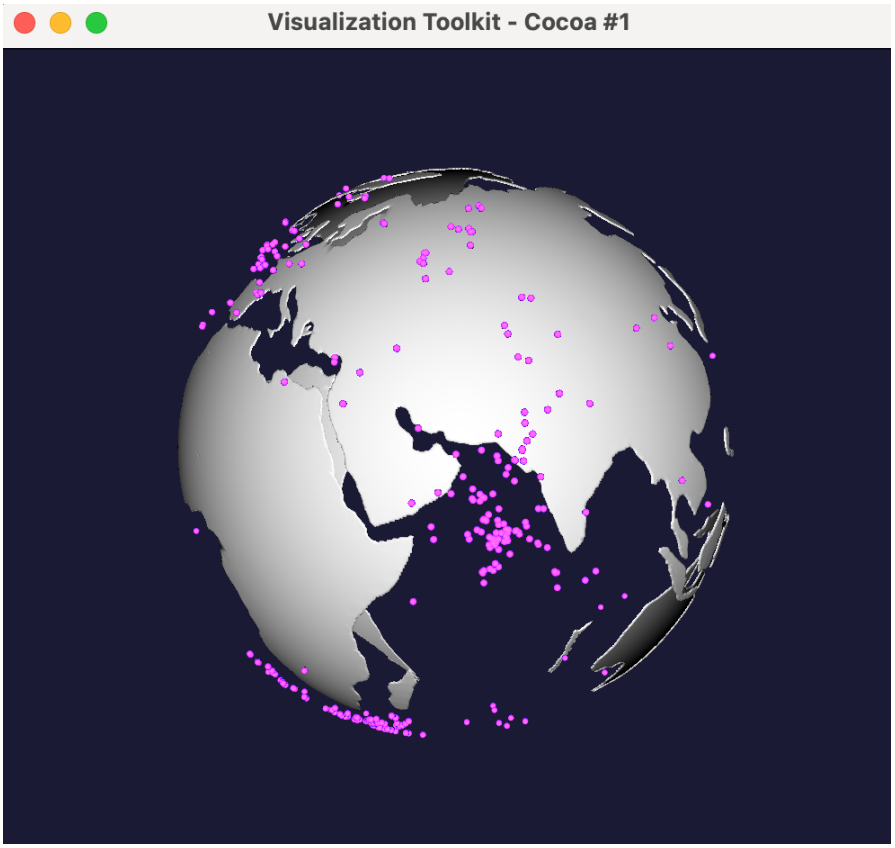
4. Rendering and Visualization
   Each location coordinates were plotted by a small red sphere using `vtkGlyph3D`.
   The renderer background and camera view are customized.
   The visualization window allows interactive zooming and scrolling through `vtkInteractorStyleTrackballCamera()`.

5. Results and Observations
   The final output displayed the globe with red dots/spheres distriduted across realistic global regions as given by *globe.obj* file. Clusters appeared around the continents which seems to align with real world IP locations and IP density.
   Some dataset rows contained invalid coordinate format (eg ZIP codes) which were excluded. Points appeared near globes surface confirming 2D to #D conversion and scaling was accurate. I have added some screenshots of the outputs below:




```
rshiju11@Rshijutas-MacBook-Air-9 Asn1_Rshijuta % /usr/local/bin/python3 /Users/rshiju11/Desktop/Asn1_Rshijuta/main.py
Center: (13.373450000000002, 8.033050000000003, -17.223599999999998), approxx. Radius: 59.10
Loaded 43652 valid coordinates from locations.dat
Total no. of 3D points plotted: 43652 points
Rendering globe with given locations:
```

```
Skipping invalid line: 54.683300 N/A 0
Skipping invalid line: 54.683300 N/A 0
Skipping invalid line: 37.386002 N/A 807
Skipping invalid line: 37.386002 N/A 807
Skipping invalid line: 37.386002 N/A 807
Skipping invalid line: 37.386002 N/A 807
Skipping invalid line: 37.386002 N/A 807
Skipping invalid line: 37.386002 N/A 807
Skipping invalid line: N/A Sydney 151.205505
Skipping invalid line: N/A Sydney 151.205505
Loaded 43652 valid coordinates from locations.dat
Total no. of 3D points plotted: 43652 points
Rendering globe with given locations:
```


Visualization Toolkit - Cocoa #1